MULTI-WAY JOINS

Cybernetically Enhanced

Salva Alcántara

Auvik

CONCEPT

```sql
INSERT INTO Y
SELECT *
FROM U1 u1
  JOIN U2 u2 ON u1.id = u2.id
  JOIN U3 u3 ON u1.id = u3.id
  ...
  JOIN UN un ON u1.id = un.id

SET 'table.optimizer.multi-join.enabled' = 'true';
```

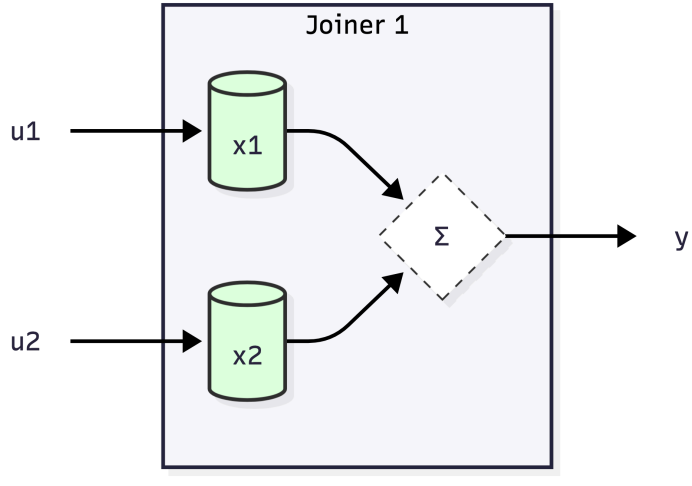
See more here.

⚠️ *Here, though, we address the DataSream API counterpart (FLINK-39131).*

MULTI-WAY JOINS

NON-MINIMAL REALIZATION

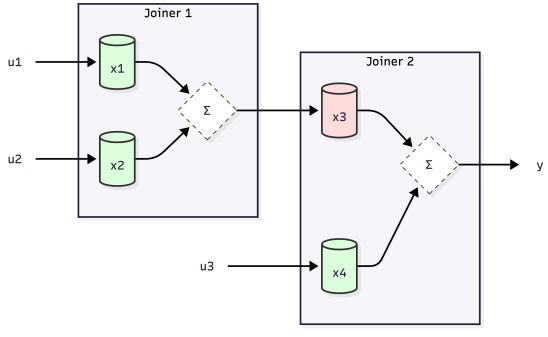
MULTI-WAY JOINS (N=2)



$$
\begin{aligned}
\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_{k+1} &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_k + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}_k \\
y_k &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_k
\end{aligned}
$$

$$
\implies y_k = u_1[k-1] + u_2[k-1]
$$

MULTI-WAY JOINS (N=3)

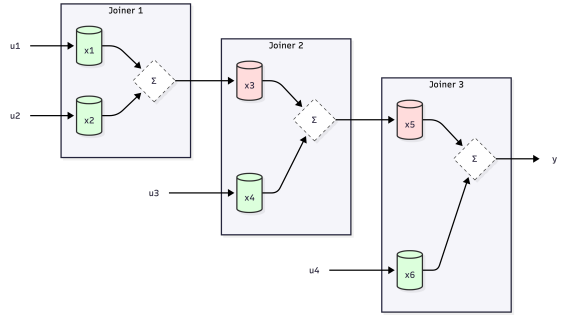


$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}_{k+1} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}_k + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}_k$$

$$y_k = \begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}_k$$

$$\implies y_k = u_1[k-2] + u_2[k-2] + u_3[k-1]$$

MULTI-WAY JOINS (N=4)



$$
\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}_{k+1} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}_k + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}_k
$$

$$
y_k = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}_k
$$

$$
\implies y_k = u_1[k-3] + u_2[k-3] + u_3[k-2] + u_4[k-1]
$$

MULTI-WAY JOINS

$$
\begin{aligned}
\mathscr{M} &= N \times S + \textstyle\sum_{i=2}^{N-1} i \times S \\
&= N \times S + \left[\frac{(N-1)N}{2} - 1\right] \times S \\
&= \frac{N^2+N-2}{2} \times S
\end{aligned}
$$

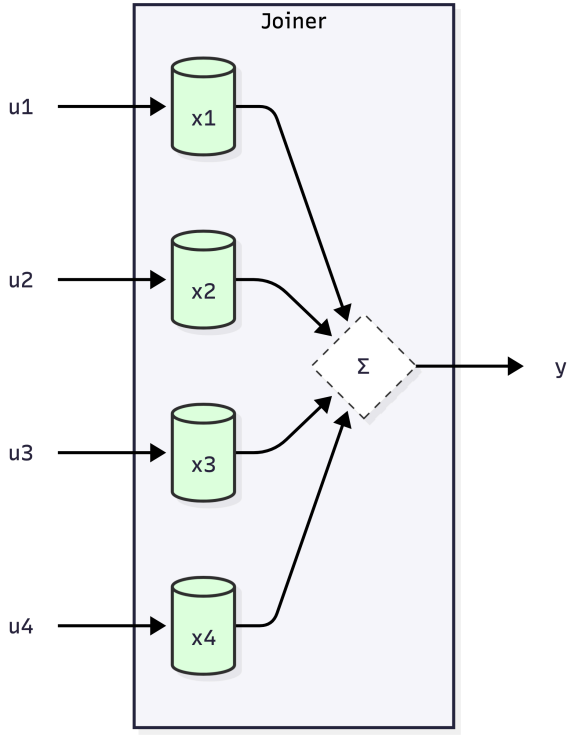
$$
\implies \mathscr{R} = \frac{N^2 + N - 2}{2N} \approx \frac{N}{2}
$$

Inputs (N)	Redundancy ($\mathscr{R}$)	Waste (%)
2	**1x**	0%
3	**1.67x**	67%
4	**2.25x**	125%
10	**5.40x**	**440%**

MULTI-WAY JOINS

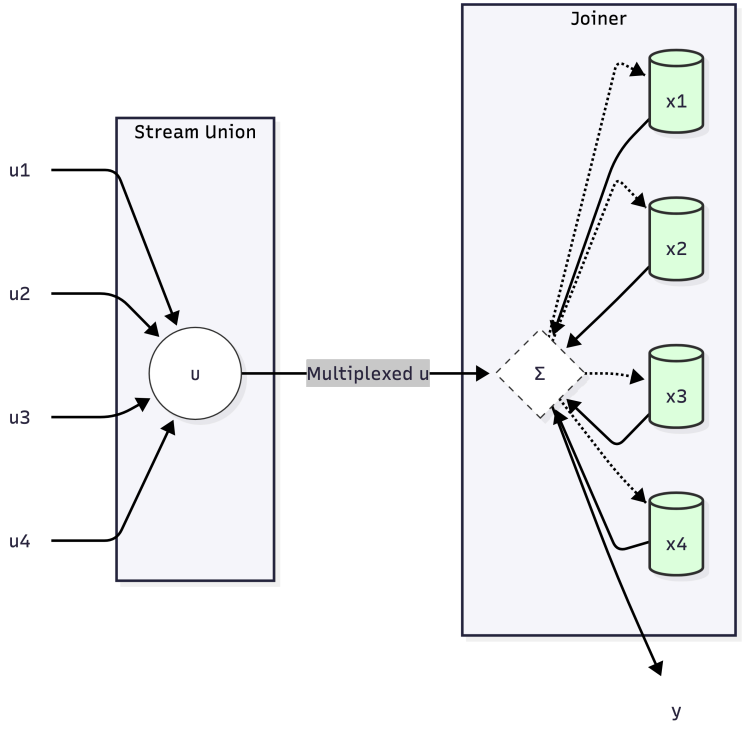
MINIMAL REALIZATION

MULTI-WAY JOINS



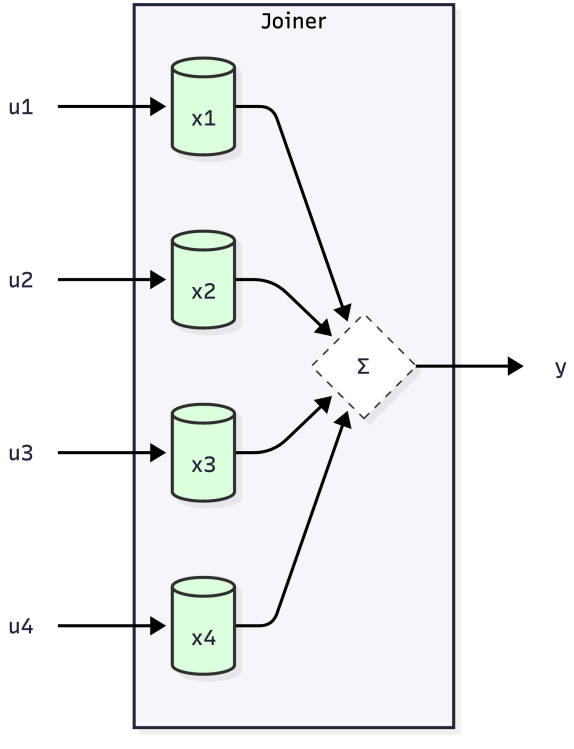
$$\begin{aligned} \mathbf{x}_{k+1} &= \mathbf{0}\mathbf{x}_k + \mathbf{I}\mathbf{u}_k \\ y_k &= \mathbf{1}^T \mathbf{x}_k \end{aligned} \implies y_k = \sum_{i=1}^{N} u_i[k-1]$$

THE—*DIY*—PATTERN

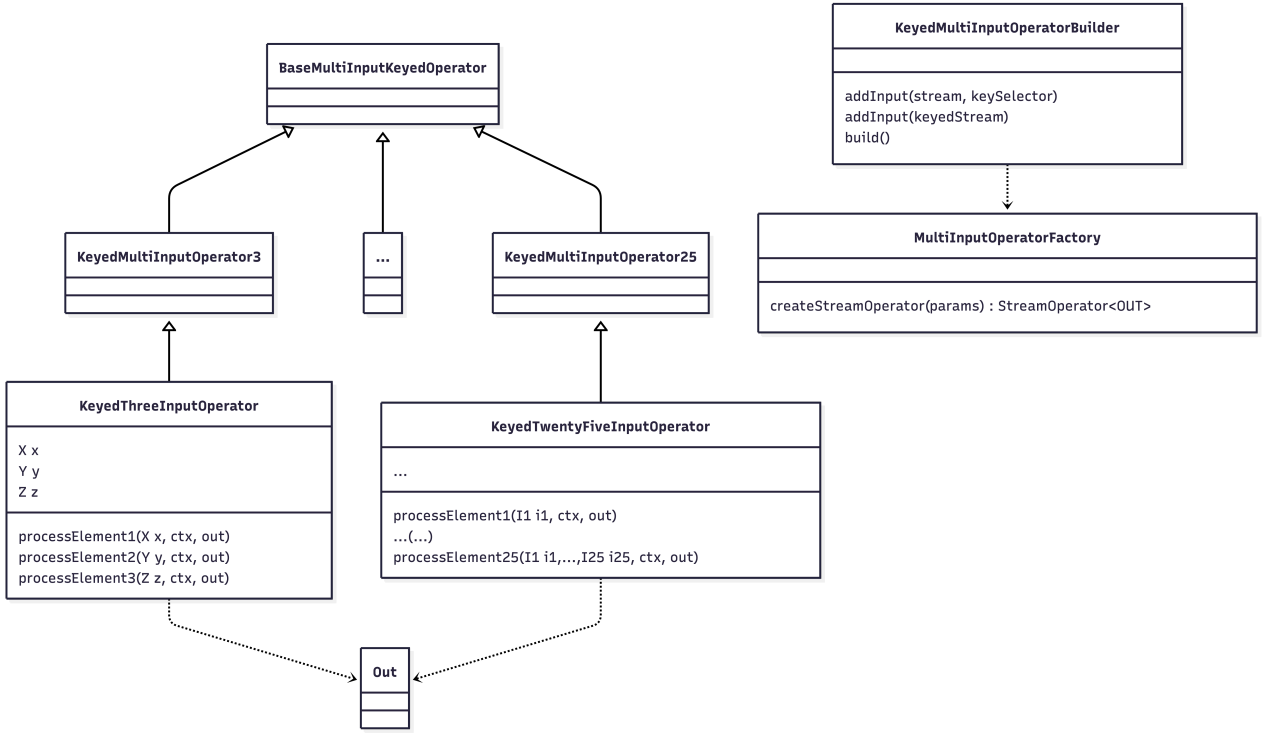


See here and here.

THE—*ERGO*—SOLUTION



THE—*ERGO*—SOLUTION



See more in FLINK-39131.

THE—*ERGO*—SOLUTION

```
KeyedMultiInputOperatorBuilder<String, Out> builder =
    new KeyedMultiInputOperatorBuilder<>(
        env,
        KeyedThreeInputOperator.class,
        TypeInformation.of(Out.class),
        Types.STRING
    );

builder
    .addInput(xs, X::getKey)
    .addInput(ys, Y::getKey)
    .addInput(zs, Z::getKey);

DataStream<Out> joined = builder.build("xyz-join");
```

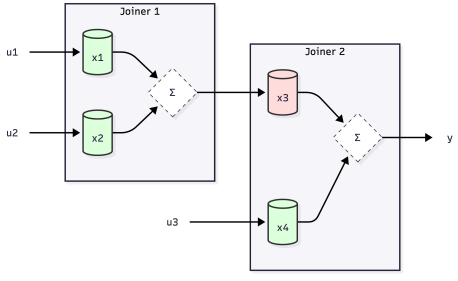
```
public class KeyedThreeInputOperator extends KeyedMultiInputOperator3<X, Y, Z, Out> {
  @Override
  protected void processElement1(X x, Context ctx, Collector<Out> out) throws Exception {
    lastX.update(x.getX());
    join(ctx, out);
  }

  @Override
  protected void processElement2(Y y, Context ctx, Collector<Out> out) throws Exception {
    lastY.update(y.getY());
    join(ctx, out);
  }

  @Override
  protected void processElement3(Z z, Context ctx, Collector<Out> out) throws Exception {
    lastZ.update(z.getZ());
    join(ctx, out);
  }
}
```

See the actual code here and here.

APPLICATION

REWRITING EXISTING JOBS

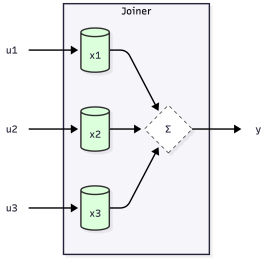
BEFORE




```
DataStream<Out> joined =
    xs.keyBy(X::getKey)
        .connect(ys.keyBy(Y::getKey))
        .process(new Joiner1())
        .keyBy(XY::getKey)
        .connect(zs.keyBy(Z::getKey))
        .process(new Joiner2());
```

```
class Joiner1 extends KeyedCoProcessFunction<String, X, Y, XY> {
  public void processElement1(X x, Context ctx, Collector<XY> out) throws Exception {
    lastX.update(x.getX());
    join(ctx, out);
  }

  public void processElement2(Y y, Context ctx, Collector<XY> out) throws Exception {
    lastY.update(y.getY());
    join(ctx, out);
  }
}

class Joiner2 extends KeyedCoProcessFunction<String, XY, Z, Out> {
  public void processElement1(XY xy, Context ctx, Collector<Out> out) throws Exception {
    lastXY.update(xy);
    join(ctx, out);
  }

  public void processElement2(Z z, Context ctx, Collector<Out> out) throws Exception {
    lastZ.update(z.getZ());
    join(ctx, out);
  }
}
```

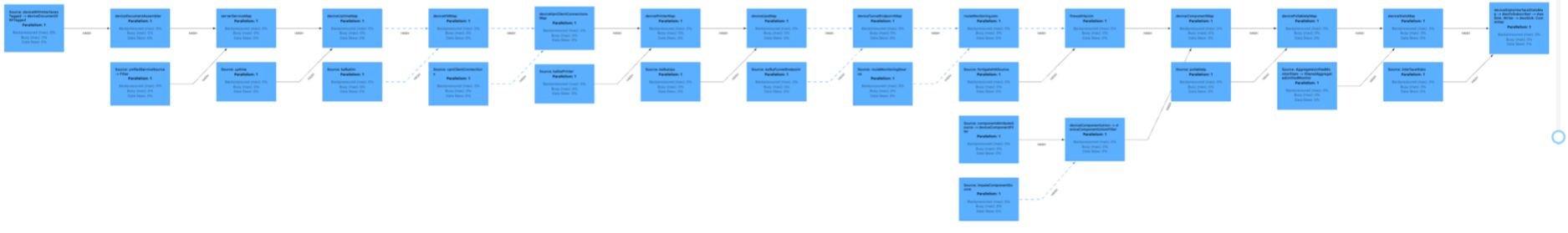
AFTER




```
KeyedMultiInputOperatorBuilder<String, Out> builder =
    new KeyedMultiInputOperatorBuilder<>(
        env,
        Joiner.class,
        TypeInformation.of(Out.class),
        Types.STRING
    );

builder
    .addInput(xs, X::getKey)
    .addInput(ys, Y::getKey)
    .addInput(zs, Z::getKey);

DataStream<Out> joined = builder.build("xyz-join");
```

```
public class Joiner extends KeyedMultiInputOperator3<X, Y, Z, Out> {
  protected void processElement1(X x, Context ctx, Collector<Out> out) throws Exception {
    lastX.update(x.getX());
    join(ctx, out);
  }

  protected void processElement2(Y y, Context ctx, Collector<Out> out) throws Exception {
    lastY.update(y.getY());
    join(ctx, out);
  }

  protected void processElement3(Z z, Context ctx, Collector<Out> out) throws Exception {
    lastZ.update(z.getZ());
    join(ctx, out);
  }
}
```

IN ACTION



⚠️ *For the most part, use AI!* 😎

WHEN TO APPLY?

*If your job has **multiple joins** sharing **a common key**, and the **intermediate state is (significanty) larger than that of the input sources**.*

That is, follow the same advice as for SQL (FLIP-516). See more here.